TASK 02 — API Gateway Security & Request Policies

Client / Postman

Sends an HTTP request

|

API Gateway — Port 8080

Routing • JWT Authentication • Authorization • Validation • Rate Limiting

|

Product Service	Order Service	User Service
8081	8082	8083

Why do we need an API Gateway

Without a gateway, clients may need to call multiple microservices directly. With a gateway, they use a common entry point where security and routing rules can be enforced.

Example:

GET http://localhost:8080/api/products

The gateway forwards the request to:

GET http://localhost:8081/api/products

The client does not need to call the Product Service directly.

Task 1: Review the existing gateway configuration

Check the gateway project, dependencies, ports, routes, security settings, and application configuration.

Expected result: You understand how the existing gateway works before modifying it.

Task 2: Configure routes for available microservices

Define which backend service receives each request.

Example: /api/products/** goes to Product Service, while /api/orders/** goes to Order Service.

Task 3: Add authentication and authorization checks

Authentication verifies who the user is. Authorization determines what the user is permitted to do.

Example: A logged-in customer can view their orders, but only an authorized administrator can access admin endpoints.

Task 4: Validate required request fields

Check whether required fields are present and valid before processing the request.

Example: An order must have a product ID and a positive quantity.

Task 5: Configure rate limits

Restrict the number of requests a client can send within a period.

Example: Allow 10 login attempts per minute per client, with appropriate account-level protections as well.

Task 6: Return appropriate HTTP error responses

Use correct HTTP status codes when requests fail.

- 400 Bad Request — Invalid request data.
- 401 Unauthorized — Missing or invalid authentication.
- 403 Forbidden — Authenticated user lacks permission.
- 404 Not Found — Resource or route does not exist.
- 429 Too Many Requests — Rate limit exceeded.
- 502 Bad Gateway — Invalid response from a backend.
- 503 Service Unavailable — Service is unavailable.

Task 7: Test valid, invalid, and unauthorized requests

Use Postman to test successful requests, missing tokens, insufficient roles, invalid data, and excessive traffic.

Task 8: Document gateway policies and test results

Record all routes, security rules, validation requirements, rate limits, HTTP responses, and test results in a README file.

Step-by-step implementation

Step 1 — Review the project

Open your project in IntelliJ IDEA or VS Code.

A possible project structure is:

```
api-gateway/
 ├── pom.xml
 ├── src/main/java/com/example/gateway/
 |       ├── ApiGatewayApplication.java
 |       ├── config/
 |       |       └── SecurityConfig.java
 |       └── filter/
 |               └── RequestValidationFilter.java
 └── src/main/resources/
         └── application.yml
```

First, inspect the existing files. Avoid replacing existing configuration before checking the current routes and dependencies.

Useful commands:

```
mvn clean
mvn test
mvn spring-boot:run
```

If Maven is installed and the project uses Maven, these commands clean the project, run tests, and start the application.

Step 2 — Configure gateway routes

Add or adapt the following in application.yml.

```
server:
  port: 8080
spring:
  application:
    name: api-gateway
```

```
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: http://localhost:8081
          predicates:
            - Path=/api/products/**
        - id: order-service
          uri: http://localhost:8082
          predicates:
            - Path=/api/orders/**
        - id: user-service
          uri: http://localhost:8083
          predicates:
            - Path=/api/users/**
```

- server.port: 8080 starts the gateway on port 8080.
- id identifies a route.
- uri specifies the backend service.
- Path determines which incoming requests match the route.

Example, a request to /api/orders/100 is forwarded to the Order Service on port 8082.

The service ports above are examples. Ensure the services actually run on those ports.

Step 3 — Add authentication and authorization

For a JWT-based setup, use Spring Security's OAuth2 Resource Server support. The gateway validates a signed JWT using a trusted issuer or public key.

Add dependencies to pom.xml for Spring Cloud Gateway, Spring Security, and the OAuth2 Resource Server. Use versions compatible with your Spring Boot and Spring Cloud release train.

Example dependency declarations:

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

Configure security for a reactive Spring Cloud Gateway application:

SecurityConfig.java

```
package com.example.gateway.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.web.server.SecurityWebFilterChain;
@Configuration
public class SecurityConfig {
   @Bean
   SecurityWebFilterChain securityWebFilterChain(
```

```
        ServerHttpSecurity http) {
    return http
        .csrf(ServerHttpSecurity.CsrfSpec::disable)
        .authorizeExchange(exchanges -> exchanges
            .pathMatchers(HttpMethod.GET, "/api/products/**")
                .permitAll()
            .pathMatchers("/api/users/**")
                .authenticated()
            .pathMatchers("/api/orders/**")
                .authenticated()
            .anyExchange()
                .authenticated()
        )
        .oauth2ResourceServer(oauth2 ->
            oauth2.jwt(Customizer.withDefaults())
        )
        .build();
  }
}
```

Configure your trusted identity provider in application.yml, using the real issuer URL for your environment:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: ${JWT_ISSUER_URI}
```

Set JWT_ISSUER_URI to your trusted identity provider's issuer URL. The identity provider must issue tokens with the expected issuer and audience, and your deployment should validate the audience as well.

Authentication is not the same as authorization. For role-based access, configure role or authority checks based on the actual claims supplied by your identity provider. Do not assume every JWT contains a particular role claim.

Step 4 — Validate required request fields

Request validation is normally implemented in the service that owns the request data. The gateway can enforce generic checks, such as content type, request size, or required headers, but it usually should not parse every business request body.

Example request DTO for Order Service:

```
package com.example.order.dto;

import jakarta.validation.constraints.Min;

import jakarta.validation.constraints.NotBlank;

public class CreateOrderRequest {

   @NotBlank(message = "Product ID is required")

   private String productId;

   @Min(value = 1, message = "Quantity must be at least 1")

   private int quantity;

   public String getProductId() {

      return productId;

   }

   public void setProductId(String productId) {

      this.productId = productId;

   }

   public int getQuantity() {

      return quantity;

   }

   public void setQuantity(int quantity) {

      this.quantity = quantity;

   }

}
```

Controller example:

```
@PostMapping("/api/orders")
public ResponseEntity<String> createOrder(
      @Valid @RequestBody CreateOrderRequest request) {
   return ResponseEntity.ok("Order request accepted");
}
```

Add spring-boot-starter-validation to the Order Service and import jakarta.validation.Valid.

A request with quantity 0 or a blank product ID should fail validation rather than being accepted.

Step 5 — Configure rate limiting

Spring Cloud Gateway supports Redis-backed rate limiting. This is useful when the gateway runs across multiple instances and needs a shared rate-limit counter.

Add the reactive Redis dependency compatible with your Spring Cloud version, and configure Redis.

Example route configuration:

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: http://localhost:8082
          predicates:
            - Path=/api/orders/**
```

```
        filters:
          - name: RequestRateLimiter
            args:
              redis-rate-limiter.replenishRate: 5
              redis-rate-limiter.burstCapacity: 10
              key-resolver: "#{@userKeyResolver}"
```

Create a rate-limit key resolver:

```
package com.example.gateway.config;
import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import reactor.core.publisher.Mono;
@Configuration
public class RateLimitConfig {
   @Bean
   KeyResolver userKeyResolver() {
      return exchange -> exchange.getPrincipal()
         .map(principal -> principal.getName())
         .switchIfEmpty(Mono.just("anonymous"));
   }
}
```

Here:

- replenishRate: 5 replenishes five tokens per second.
- burstCapacity: 10 permits a burst of up to ten tokens.
- The key resolver uses the authenticated principal to identify the caller.

For public routes, do not use one shared anonymous key for every client in production. Use a trustworthy client identity or a carefully configured IP-based resolver behind correctly configured trusted proxies. Otherwise, all anonymous callers could share the same quota, or spoofed forwarding headers could bypass the limit.

When the limit is exceeded, the gateway returns 429 Too Many Requests.

Step 6 — Return appropriate HTTP error responses

Your gateway should distinguish between authentication failures, authorization failures, validation failures, and rate-limit failures.

Typical responses:

Situation	HTTP status
Missing or invalid JWT	401
Valid user without permission	403
Invalid business request	400
Unknown route	404
Too many requests	429
Backend unavailable	503

Use Spring Security's authentication and access-denied handlers for 401 and 403, and service-level exception handling for request validation.

Example response body:

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Quantity must be at least 1"
}
```

Return only safe error details to clients. Avoid exposing stack traces, credentials, JWT contents, or internal service addresses.

Step 7 — Test with Postman

Use the following tests to verify your implementation.

Test	Request	Expected result
Public product request	GET /api/products	200 if service is available
Protected order without JWT	GET /api/orders	401
Protected order with valid JWT	GET /api/orders	200 if service is available
Invalid order quantity	POST /api/orders with quantity 0	400
Exceed rate limit	Send repeated requests to orders	429 after quota is exhausted
Unknown route	GET /api/unknown	404

For a protected request, set this header in Postman:

Authorization: Bearer YOUR_VALID_ACCESS_TOKEN

For the invalid order test, use:

```
{
  "productId": "P101",
  "quantity": 0
}
```

Expected response:

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Quantity must be at least 1"
}
```

Actual response bodies depend on your configured exception handlers.

Step 8 — Document the implementation

Create or update README.md with these sections:

API Gateway Security and Request Policies

Gateway

- Port: 8080

Routes

- /api/products/** -> Product Service
- /api/orders/** -> Order Service
- /api/users/** -> User Service

Security

- JWT authentication
- Role-based authorization where required
- Public and protected endpoints documented

Request Validation

- Required fields checked
- Invalid requests return HTTP 400

Rate Limiting

- Redis-backed rate limiting
- Order route: 5 tokens/second
- Burst capacity: 10 tokens

Testing

- Valid requests
- Invalid request bodies
- Missing and invalid JWTs
- Insufficient permissions
- Rate-limit enforcement

Results

- Record actual HTTP status codes
- Include Postman screenshots
- Document any failures and fixes